

Event Detection using Bi-LSTM Model

Antonio Gagliardi

Master Artificial Intelligence and Robotics

Sapienza, University of Rome

gagliardi.1977681@studenti.uniroma1.it

Abstract

The task of Event Detection (ED) is to identify and classify "event triggers", i.e., keywords or small parts of sentences (event chunks) that best express the occurrence of an event. The model receives as input sentences where each word (token) has been classified with a label (using the "BIO tags notation"). The model aims to learn the semantics of sentences and the classification of the different tokens according to their occurrence context to classify events in new sentences.

1 Task

There are 5 classification labels, their cardinality is $|Y| = 5$, which double for the BIO notation:

$$\forall y_i \in Y, y_{i,1} = B\text{-Label}y_i \wedge y_{i,2} = I\text{-Label}y_i$$

B-Label identifies the beginning of an event chunk, *I-Label* identifies a token within the event chunk. A special *O* symbol is added to indicate a non-event token. The entire set of classes, C , has:

$$|C| = 2 * |Y| + 1$$

1.1 Data Processing

The train.jsonl dataset is very unbalanced. Sentences contains more *O* symbols and there are some sentences that are much more longer than others. Having made these considerations and carried out a little processing it can be observed the results in Table 1:

Processed	Length	Shortest	Longest
No	20157	1	343
Yes	18531	2	156

Table 1: Train Dataset Analysis

The processing strategy is the following: When a sentence is parsed, if the punctuation mark "." occurs before the end, this is split into two distinct sentences and the algorithm continues. Another

procedure then decides which sentences discard from the dataset based on their minimum, maximum length and content (i.e. a sentence that does not contain all *O* tokens is considered more valuable).

2 Neural Architecture

The model consists of three main parts:

1. Word Embedding Layer: encode each token of the input sentence.
2. LSTM¹: Takes the sequence of encoded tokens as input and analyzes them. Its internal gate structure determines which semantic information is relevant to keep during training and which ones to forget. (Sak et al., 2014)
3. Linear Classifier: It receives the features extracted from the LSTM as input and calculates the probability of the various token labels.

2.1 Issues

The first problem is inherent in the task itself, as having to process an input sequence through an RNN type architecture it's encountered what is called the Vanish Gradient Problem. This means that during back-propagation, the gradient assumes values close to zero, and this behaviour doesn't allow the network to effectively learn something new between epochs. A second problem concerns the length of the input sentences, the longer the sentence, the more difficult it will be for the network to process the information correctly. This is avoidable using for example an N-Grams Extraction to uniform the length of the sentences. (Ramshaw and Marcus, 1995)

Ultimately, it must be considered the size of the dictionary $|V|$. Tokens that do not fit within it are encoded as <unk>.

¹<https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21>

3 NLP Toolbox

GloVe	Words	Out-Words
Glove.6b.100d	400000	4271
Glove.6b.200d	400000	4271
Glove.6b.300d	400000	4271
Glove.42b.300d	1917494	3693
Glove.840b.100d	2196016	9035

Table 2: Reading of GloVe Pre-Trained Embeddings based on vocabulary of the dataset. Out-Words denote all that words that comes outside of the GloVe vocabulary because they don't match an existing GloVe token (e.g. "firestorm" "fire-storm")

3.1 Pre-Trained Embeddings

As shown in Table 2, instead of creating a word embedding from scratch via SkipGram or CBOW networks, it's possible to resort to Pre-Trained Embeddings (e.g GloVe (Pennington et al., 2014), Fast-Text). Assuming they have been trained on very large corpus of sentences, they contain a good level of "semantic coding" between the various tokens. (It takes advantages of a good level of training of the function $P(t_x | [t_{x-i}, t_{x+i}])$, where i is the i -th token denoting the length of a window where the current token t_x appears).

3.2 N-Grams Extraction

Through this mechanism (Ramshaw and Marcus, 1995), it's performed a parsing of the input sequence x_i with a context window of length $|cw| = n$. By doing so, for each of the tokens it is obtained a context consisting of $[x_{i,j}, \dots, x_{i,n}] \forall x_i \in \text{Corpus}, \forall j = 0, \dots, |x_i|$. Thanks to this expedient it is possible to deal with the problem of excessive length of sentences by reducing the dataset to a set of N-Grams, all of the same length. Furthermore, the dataset itself is augmented.

3.3 Randomize Unknownness

A number of input sentences are randomly chosen and a n number of tokens are obscured (encoding them as $\langle unk \rangle$). Remaining the associated y label unchanged, the model tries to learn from the context that there is a given token $x_i = \langle unk \rangle$ which has a known y_i . This should help the model in the generalization during the validation and testing phase, where it is assumed a token $x_t \notin V$ may appear. (V is the vocabulary).

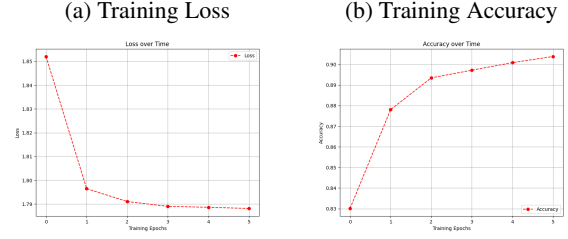


Figure 1: Loss and Accuracy after few epochs of training: at this point the model gives the best results overall the entire experiments

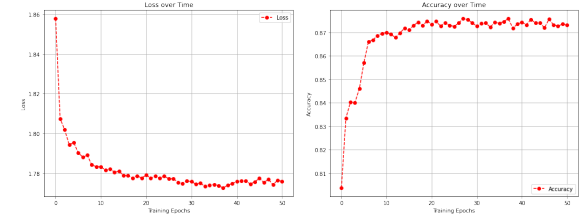


Figure 2: Loss and Accuracy after more epochs of training: continuing the training, even applying the dropout, the model continues to oscillate between the same results, also starting to suffer a degradation of performances in validation and test phases (Overfitting on the training set). This is most likely due to coping with out-words with the help of the token $\langle unk \rangle$. One solution could be to add some Char Embeddings since the dropout itself is not sufficient.

4 Training

The training was carried out using Adam as Optimizer, with a learning rate of 0.001, experimenting with batch sizes of 32 and 64. The best results were given by the batch size set to 32 (Figure 1). The latent dimension of the Word Embedding is 300 while that of the LSTM model has been set between 512 and 700, which doubles as a Bi-LSTM configuration has been adopted to guarantee the scrolling of the sentence contexts in both directions (a latent dimension greater than that of the word embedding guarantees a number of features extracted from the network on which to set the subsequent classification). A 2-layer net showed slightly better results than a single layer one, however the training slowdown added to this last consideration justifies not increasing this parameter too much. The results are shown in the Figure 2.²

²A conspicuous improvement in the results could be obtained by using the proposed architecture with the addition of a Char Embedding layer to also analyze the characters of the single token, to be able to maintain more information. (Chiu and Nichols, 2016)

Label	Precision	Recall	F1-Score
ACTION	0.7544	0.5568	0.6407
CHANGE	0.7542	0.5777	0.6543
POSSESSION	0.6360	0.5845	0.6092
SCENARIO	0.8444	0.4642	0.5990
SENTIMENT	0.7406	0.4822	0.5841
micro avg	0.7587	0.5424	0.6326
macro avg	0.7459	0.5331	0.6175
weighted avg	0.7636	0.5424	0.6318

Table 3: Evaluation Classification Report

5 Evaluating the Performances

Once the model has been trained, it's evaluated using `dev.jsonl`, Table 3, so as to decide which would be the best model to use on the `test.jsonl`. After some epoch of training the model starts show good behaviour even if it's still far from the best results.

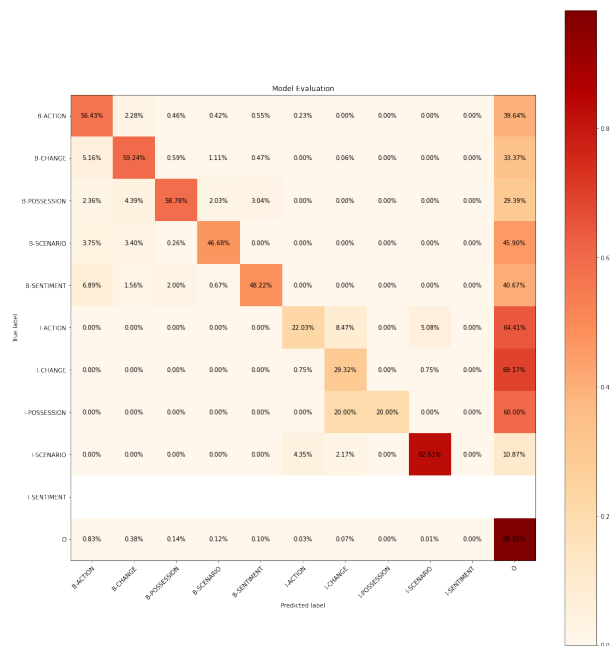


Figure 3: Confusion Matrix during Evaluation Phase

As it can be observed in Figure 3, the model failed to recognize the *I-SENTIMENT* token and is still very confused: A better result would be obtained with an increase in the percentage values in the cells of the main diagonal of the matrix, a sign that the model begins to be less confused and to align its predictions with those targets.

References

- Jason P.C. Chiu and Eric Nichols. 2016. [Named entity recognition with bidirectional LSTM-CNNs](#). *Transactions of the Association for Computational Linguistics*, 4:357–370.
- Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. [GloVe: Global vectors for word representation](#). In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
- Lance Ramshaw and Mitch Marcus. 1995. [Text chunking using transformation-based learning](#). In *Third Workshop on Very Large Corpora*.
- Haşim Sak, Andrew Senior, and Françoise Beaufays. 2014. [Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition](#).